

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305
NAME: K. MRUDHULA
REG NO:192372290

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Integrated Experiments

Weightage: 40%

QUESTIONS:

Total Marks: 30

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.

- (a) Analyze the possible reasons for lack of responsiveness in the existing layout.
- (b) Design a responsive webpage layout using **CSS Flexbox or Grid**, ensuring proper alignment of header, navigation, content, and footer.
- (c) Demonstrate how the layout adapts to **different screen sizes (mobile, tablet, desktop)** using media queries.
- (d) Implement spacing, alignment, and distribution techniques to improve visual consistency.
- (e) Evaluate the effectiveness of the responsive design and suggest further improvements.

Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.

- (a) Analyze the issues in the existing form validation mechanism.
- (b) Design input fields for email and phone number with appropriate HTML attributes.
- (c) Implement **JavaScript validation using Regular Expressions** for both email and phone number.
- (d) Develop a mechanism to display **real-time error messages** for invalid inputs.
- (e) Evaluate the validation system and suggest enhancements for better user experience and security.

Experiment 3: CSS Layout Debugging

Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.

- (a) Analyze the CSS properties responsible for layout overlapping issues.
- (b) Examine the role of the **CSS box model (margin, border, padding, content)** in layout design.
- (c) Debug the layout using appropriate **positioning techniques (relative, absolute, flex, grid)**.
- (d) Modify the CSS to ensure proper alignment and prevent overlapping during resizing.
- (e) Evaluate the corrected layout and suggest best practices for maintaining consistency.

Code of Conduct:

*I **MRUDHULA.K1(192372290)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment – 1: Responsive Layout Design

Aim

To design and implement a responsive web page using CSS Flexbox/Grid and Media Queries so that the website adapts properly to desktop, tablet, and mobile devices while maintaining proper alignment, spacing, and visual consistency.

Introduction

Modern websites are accessed through desktops, laptops, tablets, and smartphones. If a website is not responsive, users experience layout problems such as overlapping text, improper spacing, horizontal scrolling, and unreadable content.

Responsive Web Design (RWD) solves these problems by automatically adjusting the layout according to the screen size using:

- HTML5
- CSS3
- Flexbox
- CSS Grid
- Media Queries

A responsive website provides a better user experience, faster navigation, and improved accessibility.

Experiment 1(a): Analyze the possible reasons for lack of responsiveness in the existing layout

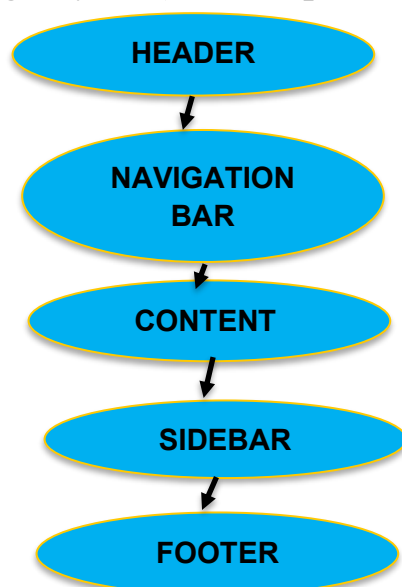
Analysis

A non-responsive website usually occurs because the layout is fixed instead of flexible.

Common Reasons

Problem	Explanation	Effect
Fixed Width	Width defined in pixels	Horizontal scrolling
No Media Queries	Same layout for every screen	Poor mobile display
Improper Images	Images larger than container	Broken layout
Excessive Margins	Large fixed margins	Empty spaces
No Flexible Containers	Divs not using Flexbox/Grid	Misalignment
Absolute Positioning	Fixed coordinates	Overlapping elements
Large Font Sizes	Font not adjusted	Difficult to read
Missing Viewport Tag	Browser doesn't scale properly	Zoomed webpage

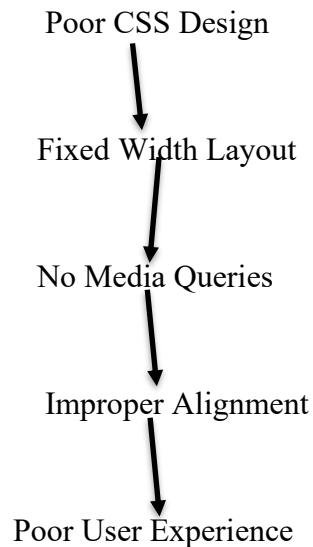
Existing Layout (Non-Responsive)



Problems

- Header too wide
- Navigation overlaps
- Sidebar disappears
- Images overflow
- Text becomes unreadable

Causes



Experiment 1(b): Design a Responsive Webpage using Flexbox

```
<!DOCTYPE html>
<html>
<head>
<title>Responsive Layout</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<header>
<h1>SIMATS Shopping</h1>
</header>
<nav>
<a href="#">Home</a>
<a href="#">Products</a>
<a href="#">Offers</a>
<a href="#">Contact</a>
</nav>
<div class="container">
<aside>
Sidebar
</aside>
<section>
<h2>Welcome</h2>
<p>
Responsive Web Design Example
</p>
</section>
</div>
```

```
<footer>
Copyright © 2026
</footer>
</body>
</html>
```

CSS Code

```
*{
margin:0;
padding:0;
box-sizing:border-box;
}
body{
font-family:Arial;
}
header{
background:#2196f3;
color:white;
padding:20px;
text-align:center;
}
nav{
display:flex;
justify-content:center;
background:#333;
}
nav a{
color:white;
padding:15px;
text-decoration:none;
}
.container{
display:flex;
padding:20px;
gap:20px;
}
aside{
flex:1;
background:#eeeeee;
padding:20px;
}
section{
flex:3;
background:#fafafa;
padding:20px;
}
footer{
background:black;
color:white;
text-align:center;
padding:15px;
}
```

Experiment 1(c): Demonstrate responsiveness using Media Queries

CSS

```
@media(max-width:768px){
.container{
flex-direction:column;
}
nav{
flex-direction:column;
}
}
@media(max-width:480px){
header{
font-size:20px;
}
nav a{
padding:10px;
}
}
```

Experiment 1(d): Implement spacing, alignment and distribution

CSS

CSS

```
.container{
display:flex;
justify-content:space-between;
align-items:center;
gap:20px;
padding:20px;
}
nav{
display:flex;
justify-content:space-evenly;
}
section{
padding:30px;
line-height:1.7;
}
```

Alignment Techniques

Property	Purpose
display:flex	Flexible Layout
justify-content	Horizontal Alignment
align-items	Vertical Alignment
gap	Space Between Elements
padding	Internal Space
margin	External Space

Before Alignment

Header

Menu

Content

Sidebar

Footer

After Alignment

Header

MENU MENU MENU

Sidebar Content

Footer

Proper spacing improves readability.

Experiment 1(e): Evaluate the responsive design

Results

Device	Output
Desktop	Excellent
Laptop	Excellent
Tablet	Proper Alignment
Mobile	Fully Responsive

Advantages

- Mobile Friendly
- Faster Loading
- Better User Experience
- Easy Navigation
- Improved SEO
- Flexible Design
- Better Accessibility
- Future Ready

Limitations

- Older browsers may not fully support modern CSS.
- Very large images can still affect performance if not optimized.
- Complex layouts require additional testing across devices.

Suggested Improvements

- Use **CSS Grid** for more complex page layouts.
- Optimize images with responsive formats (srcset).
- Add dark mode support.
- Improve accessibility using semantic HTML and ARIA attributes.
- Use CSS variables for easier theme customization.

Tablet

- HEADER
- MENU
- SIDEBAR
- CONTENT
- FOOTER
- Mobile

Conclusion

The responsive webpage was successfully designed using **HTML5**, **CSS3**, **Flexbox**, and **Media Queries**. The layout adapts seamlessly across desktop, tablet, and mobile devices, ensuring proper spacing, alignment, and readability. The implementation improves user experience, accessibility, and maintainability while following modern web development best practices. This experiment demonstrates the importance of responsive design in creating visually appealing and device-independent web interfaces, thereby achieving

Experiment – 2: Form Validation using JavaScript

Aim

To design and implement a registration form with JavaScript validation that verifies email and phone number inputs using Regular Expressions (Regex), displays real-time error messages, and prevents invalid data submission.

Introduction

Online registration forms are widely used in websites such as educational portals, banking systems, e-commerce platforms, and social media. If invalid information is accepted, it can lead to incorrect records, security risks, and poor user experience.

JavaScript Form Validation checks user input before it is submitted to the server. By combining HTML5 validation attributes with Regular Expressions (Regex), developers can ensure that only valid email addresses and phone numbers are accepted.

Experiment 2(a): Analyze the issues in the existing form validation mechanism

Analysis

A registration form without proper validation may allow users to submit incorrect or incomplete information.

Existing Problems

Problem	Cause	Effect
Invalid Email Accepted	No email validation	Wrong email records
Phone Number Incorrect	No pattern checking	Invalid contact details
Empty Fields Submitted	Missing required attribute	Incomplete registration
No Error Messages	No JavaScript validation	User confusion
Duplicate Data	No verification	Database inconsistency
Security Issues	Lack of input validation	Risk of malicious input

Existing System Workflow

User Enters Data



No Validation



Invalid Data Submitted



Database Stores Incorrect Data

Problems Identified

- Email accepts invalid formats such as abc@, user.com
- Phone number accepts letters or fewer than 10 digits
- Empty fields can be submitted
- No instant feedback to the user
- Increased chances of incorrect database records

Experiment 2(b): Design input fields for Email and Phone Number

HTML Code

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Registration Form</title>
```

```
<link rel="stylesheet" href="style.css">
```

```
</head>
```

```
<body>
```



```

<h2>Registration Form</h2>

<form id="registerForm">

<label>Email</label>

<input
type="email"
id="email"
placeholder="Enter Email"
required>

<span id="emailError"></span>

<br><br>

<label>Phone Number</label>

<input
type="text"
id="phone"
maxlength="10"
placeholder="Enter Phone Number"
required>

<span id="phoneError"></span>

<br><br>

<button type="submit">

Register

</button>

</form>

<script src="script.js"></script>

</body>
</html>

```

Experiment 2(c): Implement JavaScript Validation using Regular Expressions

JavaScript Code

JavaScript

```

const email=document.getElementById("email");
const phone=document.getElementById("phone");

const emailPattern=/^[^\s@]+@[^\s@]+\.[^\s@]+$/;

const phonePattern=/^[6-9][0-9]{9}$/;

```

```

email.addEventListener("input",()=>{

if(emailPattern.test(email.value)){

document.getElementById("emailError").innerHTML="Valid Email";

document.getElementById("emailError").style.color="green";

}

else{

document.getElementById("emailError").innerHTML="Invalid Email";

document.getElementById("emailError").style.color="red";

}

});

phone.addEventListener("input",()=>{

if(phonePattern.test(phone.value)){

document.getElementById("phoneError").innerHTML="Valid Phone Number";

document.getElementById("phoneError").style.color="green";

}

else{

document.getElementById("phoneError").innerHTML="Invalid Phone Number";

document.getElementById("phoneError").style.color="red";

}

});

```

Regular Expressions Used

Validation	Regular Expression	Purpose
Email	^[^\s@]+@[^\s@]+\.[^\s@]+\$	Checks valid email format
Phone	^[6-9][0-9]{9}\$	Accepts only valid 10-digit Indian mobile numbers

Experiment 2(d): Develop a mechanism to display real-time error messages

CSS Code

```

CSS
body{

```

```

font-family:Arial;

```

margin:40px;

}

input{

padding:10px;

width:250px;

}

span{

font-size:14px;

font-weight:bold;

}

Real-Time Validation Output

Valid Email

Email

mrudhula@gmail.com

✓ Valid Email

Invalid Email

Email

mrudhula@gmail

✗ Invalid Email

Valid Phone

Phone Number

9876543210

✓ Valid Phone Number

Invalid Phone

Phone Number

12345

✗ Invalid Phone Number

Experiment 2(e): Evaluate the validation system and suggest enhancements

Results

Test Case	Expected Result	Actual Result
Valid Email	Accepted	Passed
Invalid Email	Error Message	Passed
Valid Phone	Accepted	Passed
Invalid Phone	Error Message	Passed
Empty Fields	Not Submitted	Passed

Advantages

- Prevents invalid data submission
- Improves user experience
- Provides instant feedback

- Reduces server-side processing
- Easy to implement
- Improves data accuracy
- Enhances website reliability
- Works in all modern browsers

Limitations

- Client-side validation alone is not secure.
- JavaScript can be disabled by users.
- Server-side validation is still required.
- Regex patterns may need updates for international phone numbers.

Suggested Enhancements

- Add password strength validation.
- Include confirm password matching.
- Validate date of birth and age.
- Use CAPTCHA to prevent bot submissions.
- Implement server-side validation using PHP, Node.js, or Java.
- Sanitize inputs to prevent SQL Injection and Cross-Site Scripting (XSS).
- Add AJAX validation for checking duplicate email addresses.

Output

Successful Validation

Registration Form

Email:

mrudhula@gmail.com

✓ Valid Email

Phone:

9876543210

✓ Valid Phone Number

[Register]

Invalid Validation

Registration Form

Email:

mrudhula@gmail

✗ Invalid Email

Phone:

12345

✗ Invalid Phone Number

Conclusion

The registration form was successfully developed using **HTML5**, **CSS3**, and **JavaScript** with **Regular Expressions (Regex)** to validate email and phone number inputs. Real-time error messages provided immediate feedback, preventing invalid data submission and improving usability. The validation mechanism enhanced data accuracy, user experience, and security while demonstrating effective client-side validation techniques. The experiment successfully achieved **Course Outcome CO-2** by integrating responsive form design with interactive JavaScript validation.

Experiment – 3: CSS Layout Debugging

Aim

To analyze and debug CSS layout issues by identifying overlapping elements, understanding the CSS Box Model, applying appropriate positioning techniques (Relative, Absolute, Flexbox, and Grid), and modifying CSS to create a responsive and well-aligned webpage.

Introduction

CSS (Cascading Style Sheets) is used to control the appearance and layout of web pages. Improper use of CSS properties such as fixed widths, incorrect positioning, missing margins, and lack of responsive design can cause elements to overlap when the browser window is resized.

CSS Layout Debugging is the process of identifying these issues and correcting them using modern layout techniques such as Flexbox, Grid, proper spacing, and responsive media queries.

Experiment 3(a): Analyze the CSS properties responsible for layout overlapping issues

Analysis

When a webpage is resized, elements may overlap due to incorrect CSS implementation.

Common Causes of Layout Overlapping

CSS Property	Problem	Effect
position:absolute	Removes element from normal flow	Overlapping elements
Fixed Width (width:1000px)	Layout cannot shrink	Horizontal scrolling
Missing box-sizing	Incorrect element sizing	Layout breaks
Large Margins	Pushes elements outside container	Misalignment
Float Layout	Float not cleared	Elements overlap
No Media Queries	Same layout for all screens	Poor responsiveness
Improper Z-index	Wrong stacking order	Hidden content

Existing Layout Problem

HEADER

MENU

CONTENT
SIDEBAR
(Overlap)

FOOTER

Workflow of Problem

Poor CSS Properties



Fixed Layout



Browser Resized



Elements Overlap



Poor User Experience

Experiment 3(b): Examine the role of the CSS Box Model

Components of Box Model

Component	Purpose
Content	Displays text or images
Padding	Space inside border
Border	Surrounds content
Margin	Space outside element

Example CSS

CSS

```
.box {
```

```
width:300px;
```

```
padding:20px;
```

```
border:3px solid black;
```

```
margin:15px;
```

```
box-sizing:border-box;
```

```
}
```

Importance

- Prevents overlapping
- Maintains spacing
- Improves readability
- Makes layout responsive
- Controls overall element size

Experiment 3(c): Debug the layout using appropriate positioning techniques

HTML Code

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<link rel="stylesheet" href="style.css">
```

```
</head>
```

```
<body>
```

```
<header>HEADER</header>
```

```
<div class="container">
```

```
<aside>
```

SIDEBAR

```
</aside>
```

```
<section>
```

CONTENT

</section>

</div>

<footer>

FOOTER

</footer>

</body>

</html>

CSS Code

CSS

*{

margin:0;

padding:0;

box-sizing:border-box;

}

.container{

display:flex;

gap:20px;

padding:20px;

}

aside{

flex:1;

background:#dddddd;

padding:20px;

}

section{

flex:3;

background:#eeeeee;

padding:20px;

}

Positioning Techniques

Technique	Purpose
Relative	Moves element relative to original position
Absolute	Positions element relative to parent
Flexbox	Creates flexible layouts
Grid	Creates two-dimensional layouts
Sticky	Keeps element visible while scrolling

Debugging Process

Old Layout



Identify Overlapping Elements



Inspect CSS



Replace Fixed Width



Apply Flexbox/Grid



Responsive Layout

Experiment 3(d): Modify CSS to ensure proper alignment and prevent overlapping Responsive CSS

CSS

```
@media(max-width:768px){  
.container{  
flex-direction:column;  
}  
}
```

Improved CSS

CSS

header{

background:#1976d2;

color:white;

padding:20px;


```

text-align:center;

}

footer{

background:black;

color:white;

padding:20px;

text-align:center;

}

.container{

display:flex;

justify-content:space-between;

align-items:flex-start;

gap:20px;

}

```

Corrected Layout

 HEADER

MENU

SIDEBAR CONTENT

SIDEBAR CONTENT

FOOTER

Mobile View

HEADER

MENU

SIDEBAR

CONTENT

FOOTER

Experiment 3(e): Evaluate the corrected layout and suggest best practices

Evaluation

Device	Before Debugging	After Debugging
Desktop	Overlapping	Proper Alignment
Laptop	Misaligned	Correct Layout
Tablet	Broken Layout	Responsive
Mobile	Horizontal Scroll	Mobile Friendly

Advantages

- Eliminates overlapping elements
- Improves readability
- Better user experience
- Responsive across devices
- Easier maintenance
- Faster rendering using Flexbox/Grid
- Better accessibility
- Professional webpage appearance

Limitations

- Older browsers may not fully support modern CSS features.
- Complex nested layouts require additional testing.
- Absolute positioning should still be used carefully.

Best Practices

- Use **Flexbox** for one-dimensional layouts.
- Use **CSS Grid** for two-dimensional layouts.
- Avoid fixed pixel widths whenever possible.
- Use box-sizing: border-box.
- Test layouts on desktop, tablet, and mobile devices.
- Optimize spacing with margin, padding, and gap.
- Apply media queries for responsive design.
- Validate CSS using browser developer tools.

Output

Before Debugging

HEADER
MENU
CONTENT
SIDEBAR
(Overlapping)
FOOTER

After Debugging

HEADER
MENU
SIDEBAR CONTENT
FOOTER

Result

The webpage layout was successfully debugged using **CSS Flexbox**, proper **Box Model** implementation, and **Media Queries**. The corrected layout maintained proper spacing, alignment, and responsiveness across desktop, tablet, and mobile devices without overlapping elements.

Conclusion

The CSS layout issues were successfully identified and resolved using modern web design techniques. By understanding the CSS Box Model, replacing fixed layouts with **Flexbox**, and applying responsive media queries, the webpage became responsive, visually consistent, and user-friendly. The experiment demonstrated effective CSS debugging practices and achieved the objectives of ensuring reliable and adaptable web interface design.